

Doctor & Staff Module — Deep Dive

ဆရာဝန် နှင့် ဝန်ထမ်း မော်ဂျူး — အသေးစိတ် ရှင်းလင်းတင်ပြချက် လမ်းညွှန်

Presenter: **Ma Noo Noo Aung** (နိုင်အောင်)

Package: `com.hospitalqueue.api`

Companion to: **Queue System Team Split**

Everything needed to explain and defend this module: what each endpoint does, how a queue moves through its states, the exact code the doctor's "Call Next" button and staff's emergency confirm/reject buttons run, and a rehearsed demo script with anticipated questions.

ဤ Module အား ကောင်းမွန်စွာ ရှင်းလင်းတင်ပြနိုင်ရန်နှင့် မေးခွန်းများအား အောင်မြင်စွာ ဖြေဆိုနိုင်ရန် လိုအပ်သောအချက်အလက် အားလုံးပါဝင်ပါသည်။ — Endpoint တစ်ခုချင်းစီ၏ အလုပ်လုပ်ပုံ၊ Queue ၏ State (အဆင့်) များ ရွေ့လျားပုံ၊ ဆရာဝန်၏ "Call Next" ခလုတ်နှင့် Staff ၏ Emergency Confirm/Reject ခလုတ်များ အလုပ်လုပ်သည့် Java Code များ၊ တိုက်ရိုက် Demo ပြသရန် အစီအစဉ်နှင့် မေးလေ့ရှိသော မေးခွန်း/အဖြေများ။

- `api/DoctorApiController.java`
- `api/StaffApiController.java`
- `service/QueueService.java`
- `rule/EmergencyRule.java`
- `routes/doctor.queue.tsx`
- `routes/staff.emergency.tsx`

Where this module sits · ဤ Module ၏ စနစ်အတွင်း တည်နေရာနှင့် အခန်းကဏ္ဍ

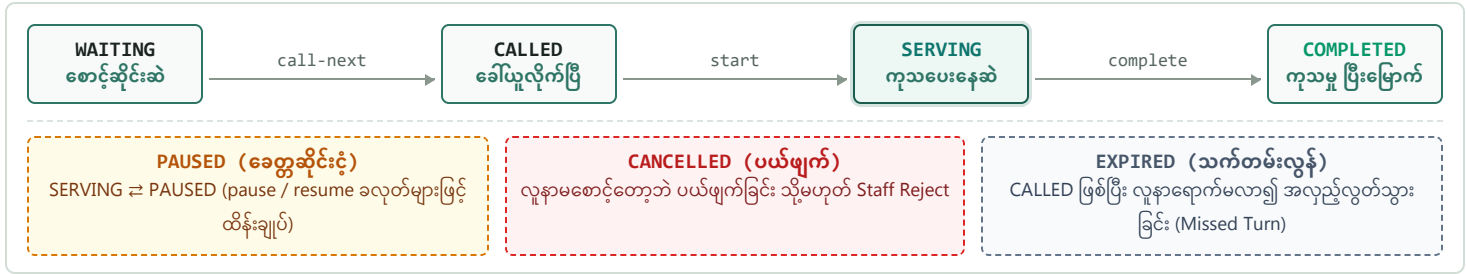
The Smart Queue Engine (Member 2 — rule/QueueRule, DoctorAvailabilityRule, EmergencyRule) decides queue order. This module is the two portals that actually **act** on that order: the doctor pulling patients out of it one at a time, and staff sitting at the front desk keeping emergency cases honest before they jump the line. Neither portal touches the database directly — both call **QueueService**, which is the single place queue state actually changes.

[မြန်မာပြန်] Smart Queue Engine (Member 2 မှ QueueRule, DoctorAvailabilityRule, EmergencyRule) သည် လူနာများ စောင့်ဆိုင်းရမည့် အစီအစဉ် (Queue Order) ကို ဆုံးဖြတ်ပေးသည်။ ဤ Member 3 Module သည် ထိုစီစဉ်ထားသော Queue အပေါ် အမှန်တကယ် လက်တွေ့အသုံးပြု လုပ်ရှားပေးသည့် **Portal (၂) ခု** ဖြစ်သည်။ — (၁) ဆရာဝန်သည် Queue ထဲမှ လူနာများကို တစ်ဦးချင်းစီ ခေါ်ယူကုသပေးခြင်း (Doctor Portal)၊ (၂) ဧည့်ကြိုကြောင်တာရှိ ဝန်ထမ်းများသည် အရေးပေါ်လူနာများ ကျော်တက်လာ ချိန်တွင် အစစ်အမှန် ဟုတ်/မဟုတ် စစ်ဆေးအတည်ပြုပေးခြင်း (Staff Portal)။ Portal နှစ်ခုစလုံးသည် Database ကို တိုက်ရိုက် Access မလုပ်ဘဲ **QueueService** ကိုသာ လှမ်းခေါ် ပြီး Queue ၏ State ပြောင်းလဲမှုအားလုံးကို QueueService တစ်ခုတည်းကသာ တာဝန်ယူဆောင်ရွက်ပါသည်။

Queue state machine · Queue အဆင့်ဆင့် ပြောင်းလဲမှု State Machine Queue.java

Every queue entry is a row moving left to right through these states. Doctor actions drive the top row; PAUSED lets a doctor step away without losing the patient's place; the bottom row are the ways a queue entry can end without being served.

(လူနာမှတ်တမ်းတိုင်းသည် ဘယ်မှညာသို့ အောက်ပါ State များအတိုင်း ကူးပြောင်းသည်။ အပေါ်တန်းသည် ဆရာဝန်၏ ကုသမှုစီးဆင်းပုံဖြစ်ပြီး PAUSED သည် ခေတ္တထွက် ခွာချိန် လူနာအလှည့် မပျောက်စေရန် ထိန်းပေးသည်။ အောက်တန်းသည် ဆေးမကုသရသေးဘဲ စာရင်းပြီးဆုံးသွားနိုင်သည့် နည်းလမ်းများ ဖြစ်သည်။)



Doctor API · ဆရာဝန်ဆိုင်ရာ API များ 7 endpoints

METHOD	PATH	WHAT IT DOES (လုပ်ဆောင်ချက် ရှင်းလင်းချက်)
GET	/api/doctor/{id}/dashboard	Dashboard အချက်အလက် အပြည့်အစုံ: called, serving, waiting လူနာစာရင်းများ၊ ယနေ့ Appointment များ၊ စာရင်းအင်း Stats များကို DB query တစ်ခုတည်းဖြင့် Batch-load ဆွဲယူပေးသည်။ Full dashboard: called/serving/waiting queues, today's appointments, stats — all patients batch-loaded in one query.
GET	/api/doctor/{id}/queue	ပေါ်ပါးသော Live Queue Endpoint: Live queue screen မှ ၈ စက္ကန့်တစ်ကြိမ် ပုံမှန်ခေါ်ယူ (Polled every 8s) သည့် ပေါ်ပါးသော Data ပုံစံ ဖြစ်သည်။ Lighter, polled every 8s by the live queue screen — same shape, fewer fields.
POST	/api/doctor/{id}/availability	တာဝန်ထမ်းဆောင်နိုင်မှု သတ်မှတ်ခြင်း: ဆရာဝန် ရရှိနိုင်မှု (Available / Unavailable) ကို အဖွင့်/အပိတ် ပြုလုပ်ခြင်း (အလုပ်ဆင်းချိန် သို့မဟုတ် နားချိန်အတွက်)။ Toggle or explicitly set available/unavailable (goes off duty / on break).
POST	/api/doctor/{id}/queue/call-next	နောက်လူနာ ခေါ်ယူခြင်း: ဦးစားပေးအမြင့်ဆုံး WAITING လူနာကို ရွေးထုတ်ကာ CALLED ဟု သတ်မှတ်သည်။ လက်ရှိ CALLED သို့မဟုတ် SERVING လူနာရှိနေပါက ခေါ်ခွင့် ငြင်းပယ်သည်။ Pulls the highest-priority WAITING patient and marks them CALLED. Refuses if someone is already CALLED or SERVING.
POST	/api/doctor/{id}/queue/start	ကုသမှု စတင်ခြင်း: လက်ရှိ CALLED ဖြစ်နေသော လူနာအား CALLED → SERVING သို့ ပြောင်းလဲပေးသည်။ CALLED → SERVING for whoever is currently called.
POST	/api/doctor/{id}/queue/complete	ကုသမှု ပြီးမြောက်ခြင်း: SERVING → COMPLETED သို့ ပြောင်းပြီး ကျန်ရှိနေသော လူနာ အားလုံး၏ စောင့်ဆိုင်းချိန် (Wait Times) ကို အလိုအလျောက် ပြန်တွက်ပေးသည်။ SERVING → COMPLETED, then recalculates wait times for everyone still waiting.
POST	/api/doctor/{id}/queue/pause · /resume	ဆိုင်းငံ့ / ပြန်လည်စတင်: လူနာအလှည့်နေရာ မပျောက်ပျက်စေဘဲ SERVING ⇄ PAUSED အပြန်အလှန် ပြောင်းလဲပေးသည်။ SERVING ⇄ PAUSED, without losing the patient's place.
GET	/api/doctor/{id}/appointments · /history · /profile	ဖတ်ရှုရန် သီးသန့် (Read-only): ဤဆရာဝန်၏ Appointment စာရင်း၊ ယခင် Queue ကုသမှုမှတ်တမ်းနှင့် Profile ကတ်။ Read-only: this doctor's appointments, past queue history, profile card.

Staff API · ညွှန်ကြားမှု/ဝန်ထမ်းဆိုင်ရာ API များ 9 endpoints (Part 1)

METHOD	PATH	WHAT IT DOES (လုပ်ဆောင်ချက် ရှင်းလင်းချက်)
GET	/api/staff/dashboard	ကောင်တာ စာရင်းအင်းများ: ယနေ့ စုစုပေါင်း၊ စစ်ဆေးရန်ကျန် အရေးပေါ်လူနာ အရေအတွက်၊ EMERGENCY/APPOINTMENT/NORMAL အချိုးအစား၊ တာဝန်ကျ vs အနားယူဆရာဝန်များ။ Front-desk stats: today's totals, emergency-pending count, EMERGENCY/APPOINTMENT/NORMAL breakdown, active vs. on-break doctors.
GET	/api/staff/emergencies	အရေးပေါ်လူနာ စာရင်း: Staff ဘက်မှ Triage စစ်ဆေးအတည်ပြုချက်ကို စောင့်ဆိုင်းနေသော Emergency-flagged လူနာစာရင်း အားလုံး။ All emergency-flagged queues awaiting staff triage confirmation.
POST	/api/staff/emergencies/{id}/confirm	အရေးပေါ်အတည်ပြုခြင်း: အစစ်အမှန်ဖြစ်ကြောင်း သတ်မှတ်သည် (emergencyConfirmed = true)၊ လူနာအား တန်းစီဇယား ရှေ့ဆုံးသို့ ရောက်ရှိကြောင်း Notification ပေးပို့သည်။ Marks the case genuine — emergencyConfirmed = true, patient notified they've moved to the front.
POST	/api/staff/emergencies/{id}/reject	အရေးပေါ်ပယ်ဖျက်ခြင်း: "Emergency rejected by staff" အကြောင်းပြချက်ဖြင့် Queue မှ ပယ်ဖျက် (Cancel) ပစ်သည်။ Cancels the queue entry with reason "Emergency rejected by staff".
GET	/api/staff/queue-monitor	တိုက်ရိုက် စောင့်ကြည့်ဘုတ်ပြား: ဆရာဝန်တစ်ဦးချင်းစီအလိုက် ကုသနေသော Ticket နံပါတ်၊ စောင့်ဆိုင်းဦးရေ၊ ပြီးမြောက်ဦးရေ (ဌာန/ဆရာဝန်အလိုက် Filter လုပ်နိုင်သည်)။ Live per-doctor board: now-serving ticket, waiting count, completed-today count — filterable by department/doctor.

METHOD	PATH	WHAT IT DOES (လုပ်ဆောင်ချက် ရှင်းလင်းချက်)
GET	/api/staff/queues/reassignable	ဆရာဝန်လွှဲပြောင်းနိုင်သော လူနာများ အခြားဆရာဝန်ထံသို့ ပြောင်းရွှေ့ပေးနိုင်သည့် WAITING Queue လူနာစာရင်းအားလုံး။ Every WAITING queue, for moving a patient to another doctor.
POST	/api/staff/queues/{id}/reassign	လူနာ လွှဲပြောင်းခြင်း Queue တစ်ခုအား အခြား ဆရာဝန်တစ်ဦးထံသို့ ရွှေ့ပြောင်းပေးသည်။ Moves one queue entry to a different doctor.
POST	/api/staff/doctors/{id}/unavailable	ဆရာဝန်အား တာဝန်မှ ခေတ္တနားစေခြင်း Staff ဘက်မှ ဆရာဝန်အား တာဝန်မှ အရေးပေါ် နားစေရန် Force ပြုလုပ်ခြင်း (ဥပမာ- ခေါ်ယူခံရခြင်း)။ Force a doctor off duty from the staff side (e.g. called away).
GET	/api/staff/appointments · /reports · /profile	ဆေးရုံတစ်ခွင် အချက်အလက်များ ဆေးရုံလုံးကျွတ် ရက်ချိန်းစာရင်း၊ ဌာနဆိုင်ရာ အစီရင်ခံစာများနှင့် ဝန်ထမ်း Profile ကတ်။ Hospital-wide appointment list, department reports, staff's own profile card.

Flagship flow: Call Next Patient · အဓိက အရေးကြီးဆုံး လုပ်ငန်းစဉ် နောက်လူနာအား ခေါ်ယူခြင်း

This is the one action most worth walking through line by line — it is where the Doctor module and the Smart Queue Engine (Member 2) actually meet.

(ဤလုပ်ငန်းစဉ်သည် တင်ပြရာတွင် တစ်ကြောင်းချင်းစီ အသေးစိတ် ရှင်းပြရန် အသင့်တော်ဆုံး Action ဖြစ်သည် — အဘယ်ကြောင့်ဆိုသော် ဆရာဝန် Module နှင့် Smart Queue Engine (Member 2) တို့ အမှန်တကယ် လက်တွေ့ဆုံဆည်း ချိတ်ဆက်သည့် နေရာဖြစ်သောကြောင့် ဖြစ်သည်။)

```
SERVICE/QUEUESERVICE.JAVA — CALLNEXT(DOCTORID)

// ၁။ လက်ရှိ လူနာအား CALLED သို့မဟုတ် SERVING ဖြစ်နေပါက နောက်လူနာ ထပ်ခေါ်ခွင့် ငြင်းပယ်သည်
if (currentConsultation(doctorId) != null)
    throw new IllegalStateException("Finish or start the current consultation first.");

// ၂။ ဤဆရာဝန်ထံ စောင့်ဆိုင်းနေသော WAITING လူနာအားလုံးကို ဆွဲထုတ်သည်
List<Queue> waiting = queueRepository.findWaitingQueuesByDoctor(doctorId);
if (waiting.isEmpty()) return null;

// ၃။ EmergencyRule ဖြင့် ဦးစားပေးအဆင့် အမြင့်ဆုံးလူနာကို ရှေ့ဆုံးသို့ ရွေးထုတ်သည်
// EMERGENCY > APPOINTMENT > NORMAL ဦးစားပေးတူပါက ရောက်ရှိသည့် မူလနံပါတ်အတိုင်း (FIFO)
Queue next = emergencyRule.reorderByPriority(waiting).get(0);
queueRepository.callNext(next.getId()); // WAITING -> CALLED
notificationService.notify(next.getId(), "It's your turn! ...");

// ၄။ နောက်တွင် ကျန်ရှိနေသော စောင့်ဆိုင်းလူနာ အားလုံးအတွက် ခန့်မှန်းစောင့်ဆိုင်းချိန်ကို Batch UPDATE ဖြင့် ပြန်တွက်သည်
updateRemainingWaitTimes(doctorId);

RULE/EMERGENCYRULE.JAVA — THE ACTUAL SORT (အမှန်တကယ် ဦးစားပေး စီပေးပုံ)

waitingQueues.sort((q1, q2) -> {
    int p1 = priorityRank(q1), p2 = priorityRank(q2); // EMERGENCY=1, APPOINTMENT=2, NORMAL=3
    if (p1 != p2) return Integer.compare(p1, p2);
    return Integer.compare(q1.getPosition(), q2.getPosition()); // ဦးစားပေးတူပါက မူလရောက်သည့်အစဉ်အတိုင်း (FIFO)
});
```

★ Good question to be ready for: "Does an emergency jump the queue before staff confirms it?" — Yes.

(ကြိုတင်ပြင်ဆင်ထားရမည့် မေးခွန်း: "Staff က Confirm မလုပ်ခင်မှာ Emergency လူနာက Queue ရှေ့ဆုံးကို ကျော်တက်သွားသလား?" — ဟုတ်ကဲ့၊ ကျော်တက်သွားပါသည်။)

အကြောင်းပြချက် ရှင်းလင်းချက်: priorityRank() သည် queue ၏ priority field တစ်ခုတည်းကိုသာ ကြည့်ပြီး sort စီတာ ဖြစ်သည်။ လူနာ (သို့မဟုတ် staff) က emergency ဟု သတ်မှတ်လိုက်သည်နှင့် ၎င်း၏ priority သည် "EMERGENCY" ဖြစ်သွားသည့်အတွက် emergencyConfirmed ဖြစ်မဖြစ် အပေါ်မမူတည်ဘဲ တန်းစီဇယား ရှေ့ဆုံးသို့ ချက်ချင်း ရောက်သွားပါသည်။ Staff confirmation သည် queue အစီအစဉ်ကို ထိန်းချုပ်တာ မဟုတ်ဘဲ၊ ကောင်တာဝန်ထမ်းက အမှုတွဲ အစစ်အမှန် ဟုတ်/မဟုတ် စစ်ဆေးပြီး မည်သူကိုင်တွယ်သည်ကို Formally Log မှတ်တမ်းတင်ပေးခြင်း ဖြစ်သည်။ လူနာအသက်အန္တရာယ် ဘေးကင်းရေးအတွက် စနစ်က အလိုအလျောက် Fast-track ဦးစားပေးထားပြီး ဖြစ်သည်။ အကယ်၍ "အရေးပေါ်အမှား (False Alarm)" ဖြစ်ခဲ့လျှင် မည်သို့လုပ်မည်နည်းဟု မေးပါက **Reject** လုပ်လိုက်ရုံဖြစ်သည် — စာရင်းကို အဆင့်နှိမ့်ချခြင်း (Demote) မျိုးမဟုတ်ဘဲ Queue မှတ်တမ်းကို လုံးဝ ပယ်ဖျက် (Cancelled Outright) ပစ်လိုက်ခြင်း ဖြစ်သည်။

Emergency confirm / reject flow · အရေးပေါ်လူနာ အတည်ပြုခြင်းနှင့် ပယ်ဖျက်ခြင်း စီးဆင်းမှု

Confirm (အတည်ပြုခြင်း)

emergencyConfirmed = true

StaffApiController.confirmEmergency →

QueueService.confirmEmergency(queueId) သည် emergencyConfirmed = true ဟု သတ်မှတ်ပြီး လူနာအား ရှေ့ဆုံးသို့ ရောက်ရှိနေပြီဖြစ်ကြောင်း Notification ပို့ပေးသည်။

✓ Log သာ မှတ်တမ်းတင်ခြင်းဖြစ်ပြီး ထပ်မံ Re-sort စီရန် မလိုပါ (ဦးစားပေးအရ ရှေ့ဆုံးသို့ ရောက်နိုင်ပြီးဖြစ်သည်။)

Reject (ပယ်ဖျက်ခြင်း)

status = CANCELLED

StaffApiController.rejectEmergency သည်

queueRepository.cancel(queueId, "Emergency rejected by staff") ကို တိုက်ရိုက်ခေါ်ယူသည်။

✓ မှတ်တမ်းသည် တန်းစီဇယားမှ လုံးဝ CANCELLED ဖြစ်သွားပြီး NORMAL သို့ အဆင့်နှိမ့်ချခြင်း (Demote) မလုပ်ပါ။

Frontend behaviour worth knowing · သိထားသင့်သော Frontend UI လုပ်ဆောင်ချက်များ

doctor.queue.tsx

setInterval 8s

getDoctorDashboard ကို ၈ စက္ကန့်တစ်ကြိမ် အလိုအလျောက် ပုံမှန်ဆွဲယူ (Polls) သည်။

data.serving သို့မဟုတ် data.called ရှိနေပါက "Call Next" ခလုတ်သည် မိမိဘာသာ Disable ဖြစ်သွားသည် — Backend တွင် ချမှတ်ထားသော "ဆရာဝန်တစ်ဦးလျှင် တက်ကြွလူနာ တစ်ဦးသာ ရှိရမည်" ဟူသော စည်းမျဉ်းကို UI တွင် တိုက်ရိုက်ရောင်ပြန်ဟပ်ထားခြင်းဖြစ်ပြီး API Reject မဖြစ်စေရန် ကြိုတင်ကာကွယ်ထားသည်။

staff.emergency.tsx

Optimistic UI

ရရှိလာသော စာရင်းတစ်ခုတည်းမှ Pending နှင့် Resolved Tabs အဖြစ် Client-side တွင် ခွဲခြားပြသသည်။

အနီရောင် "Act Now" Banner ပြသရန် Critical vs FastTrack ဟူ၍ ခွဲခြားထားသည်။ Confirm / Reject နှိပ်သည့်အခါ စာရင်းတစ်ခုလုံးကို API မှ ပြန်မဆွဲဘဲ Local State ကို အချိန်နှင့်တပြေးညီ Optimistic Patch လုပ်ပေးသည်။

Live demo script · တိုက်ရိုက် Demo ပြသရန် အစီအစဉ် ~4 min presentation

1 Open Doctor Portal → Consultation Queue

(ဆရာဝန် Portal ရှိ လူနာစောင့်ဆိုင်းခန်း မျက်နှာပြင်ကို ဖွင့်ပြပါ)

Point out the waiting list is already ordered — explain it's EmergencyRule, not arrival time.

[တင်ပြရန်] စောင့်ဆိုင်းလူနာစာရင်းသည် ရောက်လာသည့် အချိန်အလိုက် မဟုတ်ဘဲ EmergencyRule ဖြင့် ဦးစားပေး စီတန်းထားပြီးဖြစ်ကြောင်း ထောက်ပြရှင်းပြပါ။

2 Click "Call Next Patient"

("Call Next Patient" ခလုတ်ကို နှိပ်ပြပါ)

Ticket moves to the console as CALLED; note the button is now disabled — can't call two at once.

[တင်ပြရန်] Ticket နံပါတ်သည် Console ပေါ်သို့ CALLED အဖြစ် ရောက်ရှိလာပြီး၊ "Call Next" ခလုတ်သည် Disable ဖြစ်သွားပုံ (လူနာနှစ်ဦး ပြိုင်တူ မခေါ်နိုင်ပုံ) ကို ပြသပါ။

3 Click "Start Consultation" → "Complete"
("Start Consultation" နှိပ်ပြီးနောက် "Complete" ဆက်နှိပ်ပြပါ)

Show the waiting list's estimated-wait minutes shift for everyone else after Complete.

[တင်ပြရန်] Complete လုပ်လိုက်သည်နှင့် ကျန်ရှိနေသော လူနာအားလုံး၏ စောင့်ဆိုင်းချိန် (Estimated-wait minutes) များ အလိုအလျောက် လျော့ကျသွားပုံကို ပြသပါ။

4 Switch to Staff Portal → Emergency Cases
(Staff Portal ရှိ Emergency Cases စာမျက်နှာသို့ ကူးပြောင်းပြပါ)

Show a pending case in the "Pending" tab with the red Critical Cases counter.

[တင်ပြရန်] "Pending" Tab တွင် စောင့်ဆိုင်းနေသော အရေးပေါ်လူနာစာရင်းနှင့် အနီရောင် Critical Cases အရေအတွက် ပြကွက်ကို ပြသပါ။

5 Click "Confirm" on a case
(အရေးပေါ်လူနာတစ်ခုပေါ်တွင် "Confirm" ခလုတ် နှိပ်ပြပါ)

Explain: it was already first in line by priority — confirming logs staff sign-off, it doesn't re-sort anything.

[တင်ပြရန်] ၎င်းလူနာသည် ဦးစားပေးအဆင့်အရ ရှေ့ဆုံးသို့ ရောက်နိုင်ပြီးဖြစ်၍ Confirm ပြုလုပ်ခြင်းသည် Staff ဘက်မှ တာဝန်ခံအတည်ပြုကြောင်း လက်မှတ်ထိုးမှတ်တမ်းတင်ခြင်းသာဖြစ်ပြီး Re-sort ပြန်မစီကြောင်း ရှင်းပြပါ။

Likely questions · မေးလေ့ရှိသော မေးခွန်းများနှင့် အကောင်းဆုံး ဖြေဆိုချက်များ

၁။ Why can't a doctor call a second patient while one is already called?

(ဆရာဝန်တစ်ဦးသည် လူနာတစ်ဦးအား Call ထားစဉ် ဒုတိယလူနာအား အဘယ်ကြောင့် ထပ်ခေါ်၍ မရသနည်း?)

callNext() checks currentConsultation() first, which is CALLED-or-SERVING — one active consultation per doctor at a time, enforced server-side (not just the disabled button).

[မြန်မာလို ဖြေရန်] callNext() method သည် ဦးစွာ currentConsultation() ကို စစ်ဆေးပါသည်။ ၎င်းသည် CALLED သို့မဟုတ် SERVING အခြေအနေရှိ လူနာရှိမရှိ စစ်ဆေးခြင်းဖြစ်ပြီး၊ ဆရာဝန်တစ်ဦးလျှင် အချိန်တစ်ခုတွင် active consultation တစ်ခုသာ လက်ခံနိုင်စေရန် Backend Server-side တွင် တင်းကျပ်စွာ စစ်ဆေးထားသောကြောင့် ဖြစ်ပါသည် (UI ပေါ်ရှိ disabled ခလုတ်တစ်ခုတည်းဖြင့်သာ ကာကွယ်ထားခြင်း မဟုတ်ပါ)။

၂။ What happens to wait-time estimates when a patient completes?

(လူနာတစ်ဦး ကုသမှုပြီးပြောက် (Complete) သွားသောအခါ စောင့်ဆိုင်းချိန် ခန့်မှန်းချက်များ ဘာဖြစ်သွားသနည်း?)

completeCurrent() calls updateRemainingWaitTimes(), which recomputes every remaining WAITING patient's estimate as position × avg consultation minutes (plus a lunch-break offset) and writes them all in one batched UPDATE.

[မြန်မာလို ဖြေရန်] completeCurrent() သည် updateRemainingWaitTimes() ကို လှမ်းခေါ်ပါသည်။ ၎င်းသည် ကျန်ရှိနေသော WAITING လူနာတိုင်း၏ ခန့်မှန်းချိန်ကို (စောင့်ဆိုင်းအလှည့်နံပါတ် × ပျမ်းမျှ ကုသချိန်မိနစ်) ဖြင့် တွက်ချက်ကာ (နေ့လယ်စာစားချိန် အနားယူချိန်များပါ ထည့်သွင်းတွက်ချက်ပြီး) တစ်ကြိမ်တည်းသော Batched UPDATE ဖြင့် Database တွင် အားလုံးကို ပြိုင်တူ အပ်ဒိတ်လုပ်ပေးပါသည်။

၃။ Can staff move a patient to a different doctor?

(Staff ဘက်မှ လူနာအား အခြားဆရာဝန်ထံသို့ ပြောင်းရွှေ့ပေးနိုင်ပါသလား?)

Yes — /api/staff/queues/reassignable lists candidates, /api/staff/queues/{id}/reassign moves one queue entry to a different doctorId via QueueService.reassignDoctor.

[မြန်မာလို ဖြေရန်] ရပါသည်။ — /api/staff/queues/reassignable မှတစ်ဆင့် ပြောင်းရွှေ့နိုင်သော လူနာများကို ရယူနိုင်ပြီး၊ /api/staff/queues/{id}/reassign မှတစ်ဆင့် QueueService.reassignDoctor ကို အသုံးပြုကာ သက်ဆိုင်ရာ doctorId အသစ်ထံသို့ တိုက်ရိုက် လွှဲပြောင်းပေးနိုင်ပါသည်။

၄။ How does PAUSED differ from just not calling the next patient?

(PAUSED အခြေအနေသည် နောက်လူနာအား မခေါ်ဘဲ ထားခြင်းနှင့် အဘယ်သို့ ကွာခြားသနည်း?)

PAUSED is specifically for a patient already SERVING — the doctor steps away mid-consultation without marking it COMPLETED. resume() puts them back to SERVING; the queue slot isn't given up.

[မြန်မာလို ဖြေရန်] PAUSED သည် ကုသမှု စတင်နေဆဲ (SERVING) ဖြစ်နေသော လူနာအတွက် သီးသန့်ဖြစ်ပါသည်။ ဆရာဝန်သည် ကုသမှုမပြီးသေးဘဲ ခေတ္တ အရေးပေါ်ထွက်ခွာရချိန်တွင် လူနာအား COMPLETED မလုပ်ဘဲ PAUSED လုပ်ထားခြင်း ဖြစ်သည်။ ပြန်ရောက်လာပါက resume() ဖြင့် SERVING သို့ ပြန်ပြောင်းနိုင်ပြီး လူနာ၏ Queue အလှည့်အကွက် မပျောက်ပျက်စေပါ။

Presentation နှုတ်တိုက် ရှင်းလင်းတင်ပြရန် Speech Script

နိဒါန်းအတွက် စင်မြင့်ထက်တွင် ၄ မိနစ်အတွင်း ပြောဆိုတင်ပြရမည့် စကားပြောပုံစံ

တင်ဆက်မှု လမ်းညွှန်: မိမိအလှည့်ရောက်ပါက ယုံကြည်မှုအပြည့်ဖြင့် အောက်ပါ အဆင့် ၅ ဆင့်အတိုင်း မျက်နှာပြင် (Screen) ကို ထောက်ပြရင်း ရှင်းလင်းတင်ပြပါ။ စကားပြော အသံနေအထားကို သဘာဝကျကျနှင့် တိကျပြတ်သားစွာ ပြောဆိုနိုင်စေရန် ပြင်ဆင်ထားပါသည်။

နိဒါန်းအဖွင့် (၀:၀၀ - ၀:၄၅)

အဖွင့်မိတ်ဆက်ခြင်းနှင့် ဤ Module ၏ အဓိကတာဝန်အား ရှင်းပြခြင်း

"မင်္ဂလာပါ ဆရာ/ဆရာမတို့ရှင်။ ကျွန်မကတော့ Member 3 နိဒါန်းအောင် ဖြစ်ပါတယ်။ ကျွန်မ တာဝန်ယူ တင်ပြမယ့်အပိုင်းကတော့ **Doctor & Staff Module** ဖြစ်ပါတယ်။ ကျွန်မတို့ စနစ်မှာ Member 2 က Smart Queue Engine ကို အသုံးပြုပြီး လူနာတွေ ဘယ်သူ့ အရင်ကုသရမလဲဆိုတဲ့ Queue Order ကို ဆုံးဖြတ်ပေးထားပါတယ်။ ကျွန်မရဲ့ Module ကတော့ အဲဒီ Queue Order အပေါ် အမှန်တကယ် လက်တွေ့ လှုပ်ရှားဆောင်ရွက်ပေးတဲ့ Portal နှစ်ခု ဖြစ်ပါတယ်။ အဲဒါတွေကတော့ ဆရာဝန်က လူနာတွေကို တစ်ဦးချင်းစီ ခေါ်ယူကုသပေးတဲ့ **Doctor Portal** နဲ့ ညွှန်ကြားကောင်တာမှာ အရေးပေါ်လူနာတွေကို စစ်ဆေးအတည်ပြုပေးတဲ့ **Staff Portal** တို့ ဖြစ်ပါတယ်။ ဒီ Portal နှစ်ခုစလုံးဟာ Database ကို တိုက်ရိုက် မထိတွေ့တဲ့ QueueService ကနေတစ်ဆင့်သာ စနစ်တကျ ပြောင်းလဲဆောင်ရွက်ပေးပါတယ်ရှင်။"

အဆင့် ၁ (၀:၄၅ - ၁:၃၀)

Doctor Portal ဖွင့်ပြခြင်း နှင့် "Call Next" ပြသခြင်း

"ပထမဆုံးအနေနဲ့ ကျွန်မ **Doctor Portal** ရဲ့ **Consultation Queue** ကို ဝင်ရောက်ပြသပါမယ်။ ဒီနေရာမှာ တွေ့ရတဲ့ Waiting List ဟာ လူနာတွေ ဆေးရုံရောက်လာတဲ့ အချိန်အစဉ်အတိုင်း မဟုတ်ဘဲ၊ ကျွန်မတို့ရဲ့ EmergencyRule အရ Emergency ၊ Appointment နဲ့ Normal ဆိုပြီး ဦးစားပေး စီတန်းထားတာ ဖြစ်ပါတယ်။ အခု ကျွန်မ **'Call Next Patient'** ခလုတ်ကို နှိပ်လိုက်ပါမယ်။ နှိပ်လိုက်တာနဲ့ ဦးစားပေးအမြင့်ဆုံး လူနာလက်မှတ်ဟာ CALLED အဖြစ် ကောင်တာပေါ် ရောက်လာပြီး၊ လူနာဆီကိုလည်း Notification သွားပါတယ်။ တစ်ချိန်တည်းမှာပဲ 'Call Next' ခလုတ်ဟာ Disable ဖြစ်သွားတာကို တွေ့ရပါမယ်။ ဘာကြောင့်လဲဆိုတော့ ဆရာဝန်တစ်ဦးဟာ လူနာတစ်ဦးကို ခေါ်ထားချိန်မှာ နောက်လူနာ ထပ်မခေါ် နိုင်အောင် Backend ကော Frontend ကပါ တင်းကျပ်စွာ ထိန်းချုပ်ထားလို့ ဖြစ်ပါတယ်ရှင်။"

အဆင့် ၂ (၁:၃၀ - ၂:၃၀)

Start Consultation → Complete နှင့် Wait Time ပြောင်းလဲပုံ ပြသခြင်း

"လူနာ အခန်းထဲရောက်လာတဲ့အခါ **'Start Consultation'** ကို နှိပ်ပြီး CALLED ကနေ SERVING အဖြစ် ပြောင်းပါမယ်။ အကယ်၍ ဆရာဝန်က အရေးပေါ် ခေတ္တ ထွက်ခွာရ မယ်ဆိုရင် လူနာအလှည့် မဆုံးရှုံးစေဖို့ **PAUSE / RESUME** ပြုလုပ်နိုင်ပါတယ်။ ဆေးစစ်ကုသမှု ပြီးဆုံးတဲ့အခါ **'Complete'** ခလုတ်ကို နှိပ်လိုက်ပါမယ်။ အဲဒီအခါ SERVING ကနေ COMPLETED အဖြစ် အောင်မြင်စွာ ပြောင်းလဲသွားပြီး၊ နောက်မှာ စောင့်ဆိုင်းနေတဲ့ ကျန်လူနာအားလုံးရဲ့ ခန့်မှန်းစောင့်ဆိုင်းချိန် (Estimated Wait Time) တွေဟာ ချက်ချင်း အလိုအလျောက် လျော့ကျ အပိတ်ဖြစ်သွားတာကို တွေ့မြင်နိုင်ပါတယ်ရှင်။"

Staff Portal သရုပ်ပြသမှုနှင့် Code Architecture

အရေးပေါ်လူနာ ကိုင်တွယ်ပုံနှင့် Frontend-Backend ဆက်သွယ်ချက်များ

အဆင့် ၃ (2:30 - 3:30)

Staff Portal သို့ ကူးပြောင်းပြီး အရေးပေါ်လူနာ အတည်ပြုပုံ ပြသခြင်း

"ဆက်လက်ပြီး ဧည့်ကြိုကောင်တာဖြစ်တဲ့ **Staff Portal ရဲ့ Emergency Cases** စာမျက်နှာကို ပြသပါမယ်။ ဒီနေရာမှာ Pending ဖြစ်နေတဲ့ အရေးပေါ်လူနာတွေနဲ့ အနီရောင် Critical Cases Banner ကို တွေ့ရပါမယ်။ ကျွန်မ အရေးပေါ်လူနာတစ်ဦးကို **'Confirm'** လုပ်လိုက်ပါမယ်။ ဒီနေရာမှာ အရေးကြီးတဲ့ အချက်ကတော့ — ဒီလူနာဟာ စနစ်ထဲကို Emergency လို့ စာရင်းသွင်းလိုက်ကတည်းက Queue ရဲ့ရှေ့ဆုံးကို ရောက်နေပြီးသား ဖြစ်ပါတယ်။ Staff က Confirm လုပ်တာဟာ Queue ကို ထပ်ပြီး Re-sort စီတာ မဟုတ်ဘဲ၊ ကောင်တာဝန်ထမ်းက လူနာရဲ့ အရေးပေါ်အခြေအနေ အစစ်အမှန်ဖြစ်ကြောင်း အတည်ပြုပြီး ဘယ်သူကိုင်တွယ်သလဲဆိုတာ တာဝန်ခံ လက်မှတ်ထိုး Log မှတ်တမ်းတင်ပေးခြင်း ဖြစ်ပါတယ်ရှင်။ အကယ်၍ အရေးပေါ်အမှားဖြစ်ရင်တော့ Reject လုပ်ပြီး တန်းစီဇယားကနေ လုံးဝ Cancel ဖျက်ပစ်နိုင်ပါတယ်ရှင်။"

Architecture & Frontend-Backend Contract · စနစ်တည်ဆောက်ပုံနှင့် ချိတ်ဆက်ပုံ အဓိကအချက်များ

Backend Enforcement (Java Spring Boot)

- **Single Source of Truth:** Queue ၏ အခြေအနေပြောင်းလဲမှု မှန်သမျှသည် QueueService.java တစ်ခုတည်းတွင်သာ ဖြစ်ပေါ်သည်။
- **Strict Invariant:** ဆရာဝန်တစ်ဦးလျှင် အချိန်တစ်ခုတွင် active consultation (CALLED သို့မဟုတ် SERVING) တစ်ဦးသာ ရှိစေရန် Controller နှင့် Service အဆင့် တွင် စစ်ဆေးသည်။
- **Batched Performance:** လူနာတစ်ဦး Complete ဖြစ်တိုင်း စောင့်ဆိုင်းလူနာ အားလုံး၏ wait time ကို SQL Batch Update ဖြင့် တစ်ပြိုင်နက် သိမ်းဆည်းသည်။

Frontend Experience (React / Vite)

- **Lightweight Polling:** doctor.queue.tsx သည် ၈ စက္ကန့်တစ်ကြိမ် လိုအပ်သော data သာပါသည့် ဝေပါးသော API ကို ခေါ်ယူသည်။
- **Button State Mirroring:** Active consultation ရှိနေချိန်တွင် Call Next ခလုတ်အား UI ပေါ်တွင် disable လုပ်ထားသဖြင့် API Reject မဖြစ်စေရန် ကြိုတင်ကာကွယ်သည်။
- **Optimistic Updates:** staff.emergency.tsx တွင် Confirm/Reject နှိပ်ပါက စာရင်းတစ်ခုလုံး အသစ်ပြန်မခေါ်ဘဲ Local State ကို ချက်ချင်း အပ်ဒိတ်လုပ်သည်။

Flagship Flow: Step-by-Step Execution Lifecycle · အဆင့် ၄ ဆင့်ဖြင့် လည်ပတ်ပုံ

- ၁။ **Validation:** currentConsultation(doctorId) != null စစ်ဆေးပြီး ရှိနေပါက IllegalStateException ထုတ်ကာ နောက်လူနာ ထပ်ခေါ်ခွင့် ငြင်းပယ်သည်။
- ၂။ **Fetching:** သက်ဆိုင်ရာ ဆရာဝန်ထံ တန်းစီစောင့်ဆိုင်းနေသော WAITING လူနာများအားလုံးကို ဆွဲထုတ်သည်။
- ၃။ **Sorting & Notification:** emergencyRule.reorderByPriority() ဖြင့် ဦးစားပေးအဆင့် (EMERGENCY=1, APPOINTMENT=2, NORMAL=3) အလိုက် စီပြီး ရှေ့ဆုံးလူနာအား CALLED အဖြစ် ပြောင်းကာ လူနာထံ Notification ပေးပို့သည်။
- ၄။ **Wait-Time Recomputation:** ကျန်ရှိသော စောင့်ဆိုင်းလူနာအားလုံးအတွက် စောင့်ဆိုင်းချိန်အား ချက်ချင်း ပြန်လည်တွက်ချက်ကာ Database တွင် အစုလိုက် သိမ်းဆည်းသည်။

မေးခွန်းများအား အမှတ်ပြည့်ရစေမည့် နှုတ်ဖြေနည်းဗျူဟာများ

ဆရာ/ဆရာမများ၏ မေးခွန်းများအား အထူးယုံကြည်မှုရှိရှိ ဖြေဆိုနိုင်ရန် လမ်းညွှန်ချက်

စာမေးပွဲတွင် ဆရာ/ဆရာမများသည် သင်၏ Code နှင့် Architecture ကို အမှန်တကယ် နားလည်သဘောပေါက်မှု ရှိ/မရှိ စစ်ဆေးလေ့ရှိပါသည်။ အောက်ပါ မေးခွန်းနှင့် ဖြေဆိုပုံ နည်းဗျူဟာများကို ကြိုတင်လေ့ကျင့်ထားပါ။

မေးခွန်း ၁: "DoctorApiController ထဲက Dashboard Endpoint နဲ့ Queue Endpoint က ဘာကွာခြားလို့ နှစ်ခု ခွဲထားတာလဲ?"

အကောင်းဆုံး ဖြေဆိုချက်: "ဒါက Performance Optimization ကြောင့် ဖြစ်ပါတယ် ဆရာ/ဆရာမရှင်။ /dashboard endpoint ကတော့ ဆရာဝန် စာမျက်နှာ စတင်ဖွင့်ချိန်မှာ appointments တွေ၊ ပြီးမြောက်ခဲ့တဲ့ stats တွေအပါအဝင် အချက်အလက် အကုန်လုံးကို အပြည့်အစုံ ဆွဲယူတာ ဖြစ်ပါတယ်။ ဒါပေမဲ့ /queue endpoint ကတော့ Live Queue မျက်နှာပြင်ကနေ စ တက္ကန့်တစ်ကြိမ် မကြာခဏ ပုံမှန်ခေါ်ယူ (Polling) လုပ်နေတဲ့အတွက် မလိုအပ်တဲ့ appointment နဲ့ stat တွေကို ဖယ်ထုတ်ပြီး လက်ရှိ တန်းစီနေတဲ့ လူနာစာရင်း သီးသန့်ကိုသာ Database round-trip အနည်းဆုံးနဲ့ ပေါ့ပေါ့ပါးပါး ပြန်ပို့ပေးနိုင်အောင် သီးခြား ခွဲထုတ်ထားတာ ဖြစ်ပါတယ်ရှင်။"

မေးခွန်း ၂: "လူနာတစ်ဦးဟာ တန်းစီနေရင်း ဆရာဝန်ပြောင်းချင်တယ်ဆိုရင် စနစ်က ဘယ်လို ကိုင်တွယ်သလဲ?"

အကောင်းဆုံး ဖြေဆိုချက်: "Staff Portal မှာ ပါဝင်တဲ့ /api/staff/queues/reassignable ကနေတစ်ဆင့် လက်ရှိ WAITING ဖြစ်နေတဲ့ လူနာစာရင်းကို ရယူနိုင်ပြီး၊ /api/staff/queues/{id}/reassign endpoint ကို သုံးကာ QueueService.reassignDoctor() နဲ့ အခြားဆရာဝန်ထံသို့ တိုက်ရိုက် လွှဲပြောင်းပေးနိုင်ပါတယ်။ ဒီအခါ မူလ ဆရာဝန်ရဲ့ စောင့်ဆိုင်းချိန်ရော ဆရာဝန်အသစ်ရဲ့ စောင့်ဆိုင်းချိန်ပါ အလိုအလျောက် ပြန်လည်တွက်ချက် ညီညွတ်သွားစေပါတယ်ရှင်။"

မေးခွန်း ၃: "Frontend မှာ ခလုတ်ကို Disable လုပ်ထားတာ ရှိပြီးသားကို Backend Controller မှာ ဘာလို့ ထပ်စစ်နေသေးတာလဲ?"

အကောင်းဆုံး ဖြေဆိုချက်: "Frontend က Disable လုပ်ထားတာဟာ UI/UX အရ အသုံးပြုသူ အမှားမဖြစ်အောင် ကာကွယ်ပေးတဲ့ Client-side Validation သာ ဖြစ်ပါတယ်။ ဒါပေမဲ့ တကယ့် System Security နဲ့ Data Integrity အတွက် Backend Server-side မှာ မဖြစ်မနေ ထပ်မံ စစ်ဆေးရပါမယ်။ အကယ်၍ တစ်စုံတစ်ယောက်က Postman သို့မဟုတ် cURL ကနေ API ကို တိုက်ရိုက် လှမ်းခေါ် ခဲ့ရင်တောင် Backend က 'Finish or start current consultation first' ဆိုပြီး တင်းကျပ်စွာ ငြင်းပယ်နိုင်အောင် ကာကွယ်ထားခြင်း ဖြစ်ပါတယ် ရှင်။"

မေးခွန်း ၄: "PAUSED အခြေအနေမှာ ရှိနေတဲ့ လူနာကို ဆရာဝန်က မေ့ပြီး နောက်လူနာ ထပ်ခေါ်လို့ ရနိုင်သလား?"

အကောင်းဆုံး ဖြေဆိုချက်: "လုံးဝ မရနိုင်ပါရှင်။ callNext() စစ်ဆေးတဲ့ currentConsultation() logic မှာ PAUSED ဖြစ်နေရင်တောင် အဆိုပါ လူနာဟာ တာဝန်ခံဆရာဝန်ထံ မှာ Consultation မပြီးပြတ်သေးဘဲ ဆိုင်းငံ့ထားတာဖြစ်တဲ့အတွက် နောက်ထပ် လူနာသစ် ခေါ်ယူခွင့်ကို Backend က တားမြစ်ထားပါတယ်။ အဲဒီလူနာကို Resume လုပ်ပြီး Complete ဖြစ်အောင် ကုသပြီးမှသာ နောက်လူနာကို ခေါ်ယူနိုင်မှာ ဖြစ်ပါတယ်ရှင်။"